

In the above code, in Line No. 11, we are trying to change the value of the variable to which the pointer is 'pointing to', but this is not possible since the value is constant. When the above code is compiled and run, we get the output shown in Figure 1.

Illustration 2

Now, let's use the same example given in Illustration 1 to show that the 'address' that the pointer contains is not a constant.

```
satya@satya:~$ gcc pointer_const.c
pointer_const.c: In function 'main':
pointer_const.c:12:2: error: assignment of read-only location '*ptr'
  ptr = 30; //Error : Since, the pointer pointing to the value is constant.
  ^
satya@satya:~$
```

Figure 1: Output of the code snippet given in Illustration 1

```
1 #include <stdio.h>
2
3 int main()
4 {
5     //Definition of the variables
6     int a = 10;
7     int b = 20;
8
9     //Definition of pointer to constant
10    const int* ptr = &a; //Now, ptr is pointing to
    the value of the variable 'a'
11
12    ptr = &b; // Works: Since pointer is not constant
13
14    return 0;
15 }
```

From Illustrations 1 and 2, one can understand that the 'address' that the pointer contains can be changed but not the value to which the pointer is 'pointing to'. This can be clearly understood by the pictorial representations given in Figures 2, 3 and 4.

Constant pointers

A 'constant pointer' is one that cannot change the address it contains. In other words, we can say that once a constant pointer points to a variable, it cannot point to any other variable.

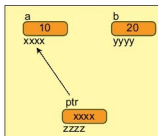


Figure 2: Pictorial representation of 'pointer to constant'

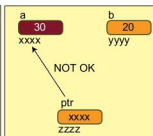


Figure 3: Output of the code snippet given in Illustration 3

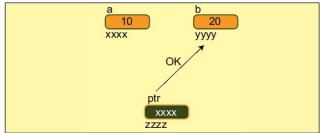


Figure 4: Pictorial representation of 'constant pointer'

Table 2: Pointer to constant concept

Pointer to constant	Value change	Address change
Const int*ptr;	Not possible	Possible



Note: However, these pointers can change the value of the variable they 'point to' but cannot change the address they are 'holding'.

Table 3: Showing how to declare 'constant pointer'

Syntax	Example
<type of pointer>'const <pointer name>	int*const ptr

Table 4: Constant pointer concept

Pointer to constant	Value change	Address change
int*const ptr;	Possible	Not possible

Table 5: Summary

Example	Value constant	Pointer constant
char*ptr	No	No
const char*ptr	Yes	No
char const*ptr	Yes	No
char*const ptr	No	Yes
const char*const ptr	Yes	Yes

Illustration 3

Let us consider the following code snippet to understand how 'constant pointer' works:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     //Definition of the variable
6     int a = 10;
7     int b = 20;
8
9     //Definition of pointer to constant
10    const int* ptr = &a; //Now, ptr is pointing to the
    value of the variable 'a'
11
12    *ptr = 30; // Works, since the pointer pointing to
```